CS 3563: Introduction to DBMS II

Homework 5: Transaction Management

Name: Pathri Vidya Praveen
Roll No.: CS24BTECH11047
Due Date: 26 March 2026, 11:59 PM IST

Question 1

Lock Compatibility Matrix

To determine the lock compatibility matrix, we map each operation to its underlying Read (R) or Write (W) primitive and apply the fundamental conflict rule:

> Two operations conflict if they access the same object and at least one of them is a Write.

Operation Mapping

- $R(a, v)$: Pure **Read** operation.
- $S(a, v)$: Pure **Write** operation (replaces the value).
- $D(a, v)$: **Read–Write** operation (reads the current balance, computes the new value, then writes it).
- $W(a, v)$: **Read–Write** operation (reads the current balance, computes the new value, then writes it).

Lock Compatibility Matrix

	$R(a,v)$	$S(a,v)$	$D(a,v)$	$W(a,v)$
$R(a,v)$	Yes	No	No	No
$S(a,v)$	No	No	No	No
$D(a,v)$	No	No	No	No
$W(a,v)$	No	No	No	No

Justification

1. R **vs.** R**: Compatible**
   Both operations are reads and do not modify the balance. Hence, no conflict occurs.
2. R **vs.** $\{S, D, W\}$**: Incompatible**
   Operations S, D, and W perform writes. Since one operation is a read and the other is a write, they conflict.
3. S **vs. Any Operation: Incompatible**
   $S(a, v)$ overwrites the balance. Therefore:

- With R: write–read conflict
- With S, D, W: write–write conflict

4. D **and** W **vs. Any Operation: Incompatible**
   Both D and W perform read–modify–write actions:

   (a) Read the current balance

   (b) Compute the new balance

   (c) Write the updated value

   Thus:

   - With R: write–read conflict
   - With S, D, W: write–write conflict

Conclusion

Since S, D, and W all perform write operations, they require exclusive access. Only R can share a lock with another R, as it is the only pure read operation. Consequently, all entries in the matrix are incompatible except the (R, R) pair.

Question 2

Based on the provided partial schedule S, we analyze its properties regarding conflict-serializability and the Two-Phase Locking (2PL) protocol.

$$S : R_2(d), R_1(a), W_1(a), R_3(b), R_3(c), R_2(a), W_2(a), R_2(c), W_3(c), R_1(d), R_3(d), W_1(d)$$

1. Conflict-Serializability Analysis

A schedule is conflict-serializable if and only if its precedence graph is acyclic. A conflict occurs between two operations belonging to different transactions on the same data item where at least one operation is a **Write**.

Step 1: Identification of Conflicts

Tracing the data items in chronological order:

- **Item** a**:** $W_1(a)$ precedes $R_2(a)$ and $W_2(a)$. This creates the edge: $T_1 \to T_2$.
- **Item** c**:** $R_2(c)$ precedes $W_3(c)$. This creates the edge: $T_2 \to T_3$.
- **Item** d**:**
  - $R_2(d)$ precedes $W_1(d)$. This creates the edge: $T_2 \to T_1$.
  - $R_3(d)$ precedes $W_1(d)$. This creates the edge: $T_3 \to T_1$.

Note: Read–Read pairs (e.g., $R_2(d)$ and $R_3(d)$) do not conflict and are ignored in graph construction.

Step 2: Precedence Graph and Conclusion

The resulting precedence graph contains the edges $\{(T_1, T_2), (T_2, T_3), (T_2, T_1), (T_3, T_1)\}$. The presence of the cycle $T_1 \rightarrow T_2 \rightarrow T_1$ confirms that the schedule is **not conflict-serializable**.

2. Feasibility under 2-Phase Locking (2PL)

The schedule **cannot** be produced by the Basic 2PL protocol for the following reasons:

1. **Theorem Violation:** A fundamental property of 2PL is that it only generates conflict-serializable schedules. Since this schedule is non-serializable, 2PL cannot produce it.
2. **Phase Violation:** Under 2PL, a transaction enters the *Shrinking Phase* as soon as it releases its first lock, after which it **cannot acquire any new locks**.
    - For T_1 to execute $W_1(a)$, it must hold an exclusive lock $X_1(a)$.
    - For T_2 to later perform $R_2(a)$, T_1 must have already released $X_1(a)$.
    - However, later in the schedule, T_1 performs $R_1(d)$ and $W_1(d)$. This would require T_1 to acquire a new lock on d *after* it has already released a lock on a.

Verdict: The schedule is neither conflict-serializable nor capable of being generated by a 2PL-compliant scheduler.

Question 3

Response:

The vendor's guarantee that the hardware and software will never fail eliminates only the possibility of external system failures (crashes). However, a database management system must also handle **transaction failures**. Therefore, removing the log-based recovery manager is not advisable, as it is essential for maintaining the ACID properties, especially **Atomicity**.

1. Transaction Failure vs. System Failure

A permanently operational system does not prevent individual transactions from failing. Transactions may abort for several reasons, including:

- **Logical errors:** A transaction detects an internal condition that prevents successful completion (e.g., insufficient funds).
- **System-induced aborts:** The DBMS may abort a transaction to resolve a deadlock in concurrency control.
- **User intervention:** A user or application may explicitly issue a `ROLLBACK` or cancel execution.

In all such cases, partial updates may already have been performed. The recovery manager uses the log to ensure **Atomicity** — that a transaction is executed as an "all-or-nothing" unit. Without recovery, these partial changes could remain in the database, leading to an inconsistent state.

2. Effect of Update Policy

The necessity of recovery does not disappear; only its mechanism differs depending on when updates are written to disk.

Immediate Updates (UNDO Required)

In immediate update schemes, database pages may be written to disk before the transaction commits.

- Uncommitted changes can reach disk.
- If the transaction aborts, these changes must be reversed.
- The recovery manager uses UNDO information in the log to restore the previous values.

Without recovery, aborted transactions would leave incorrect data on disk, violating atomicity and corrupting the database.

Deferred Updates (No Disk UNDO Required)

In deferred update schemes, updates are applied to the database only after the transaction commits.

- Uncommitted changes never reach disk.
- If a transaction aborts, its pending updates are simply discarded.
- Disk UNDO is therefore unnecessary.

However, the system must still track transaction status and ensure that only committed transactions modify the database. Thus, recovery logic remains necessary to manage transaction lifecycles and maintain correctness.

Conclusion

Even with a guarantee of no system failures, the recovery manager cannot be removed because it is required to handle transaction failures and preserve atomicity.

- Immediate update systems require UNDO to roll back aborted transactions.
- Deferred update systems avoid disk UNDO but still require recovery mechanisms to manage commits and aborts.

Therefore, unless it is guaranteed that no transaction will ever abort, the recovery manager must remain part of the DBMS. In summary, while the vendor guarantee removes the need for crash recovery (Redo-heavy), the Recovery Manager remains indispensable for Transaction Recovery (Undo-heavy in immediate systems) to prevent database corruption from aborted transactions.

Question 4

Answer:

1. Recoverability of the System

Yes. The system provides recoverability because it satisfies the two fundamental requirements of log-based recovery.

WAL Rule 1 (Atomicity): Log records for updates are flushed to stable storage before the corresponding data pages are written to disk. This guarantees that the *before-image* of every update is available in the log, allowing the system to undo partial effects of uncommitted transactions.

WAL Rule 2 (Durability): At commit time, all updated pages are forced to disk and a commit record is written to the log. Once the commit record reaches stable storage, the transaction is considered durable.

2. Crash Scenario Before Commit Record

Consider a crash that occurs after updated data pages have been written to disk but before the commit record is logged.

State: The database on disk reflects the updates, but the log contains no $\langle T_i \text{ commit} \rangle$ record.

Recovery Action: During recovery, T_i is classified as a *loser* transaction because it has a start record but no commit record. Since the log record containing the old value was flushed before the data write (due to WAL), the recovery manager performs an **UNDO** to restore the previous values.

Result: The database is returned to its consistent state prior to the execution of T_i, thereby preserving atomicity.

3. Behavior of UNDO and REDO Operations

This system effectively follows a **Steal / Force** buffer management policy.

UNDO Operations (Required)

Necessity: High.

Action: The recovery manager scans the log backward to identify all loser transactions (those with a start record but no commit or abort record). Using the before-images stored in the log, it restores the old values on disk.

Reason: Because the system allows pages with uncommitted updates to be written to disk (Steal policy), the disk may contain partial effects of incomplete transactions. UNDO removes these effects to restore consistency.

REDO Operations (Minimal)

Necessity: Low (but logically part of recovery).

Action: The recovery manager scans the log forward to identify winner transactions (those with commit records) and reapplies their updates if necessary.

Reason: Under the Force policy, all updated pages of committed transactions are written to disk before the commit record is logged. Therefore, the disk already reflects the committed state, and REDO typically performs little or no actual work.

Summary Verdict

The classmate's system implements a **Steal / Force** recovery strategy. Although this approach incurs higher disk I/O overhead at commit time, it is logically sound and fully recoverable.

- UNDO is required to remove effects of uncommitted transactions.
- REDO is generally minimal because committed updates are forced to disk at commit.

Therefore, the system provides correct recoverability.